

EdgePad — Building Trackpad Edge Gestures on macOS 27

A complete, step-by-step build guide

Target: macOS 27 “Golden Gate” · Xcode 27 · Swift 6.4 · Apple silicon only

0. What you are building, and what stands in your way

EdgePad is a menu-bar utility (no Dock icon, no window) that watches raw finger positions on your trackpad. When a finger starts inside one of four narrow bands along the trackpad edges and then slides *along* that band, EdgePad fires a repeating action.

Edge	Finger travels	Default action
Right	up / down	Volume up / down
Top	right / left	Brightness up / down
Left	up / down	(your choice — stage 3)
Bottom	right / left	(your choice — stage 3)

The four things that make this project non-trivial

Read these now. Each one will otherwise cost you an evening.

- macOS gives you normalized touch coordinates, but only through gesture events.** `NSTouch.normalizedPosition` returns `(x, y)` in `0...1` with the origin at the **bottom-left** of the trackpad. This is exactly what you need. The catch is that it arrives on `NSEvent` objects of type `.gesture`, which only reach your app when your app is frontmost — unless you install a `CGEventTap`.
- A system-wide event tap cannot be sandboxed.** `CGEvent.tapCreate` is blocked inside the App Sandbox. So: App Sandbox **off**, and the app can never ship on the Mac App Store. Distribution is Developer ID + notarization, or just local use.
- A single finger sliding on the trackpad also moves the cursor.** The system generates `mouseMoved` events independently of the touch data. If you ignore this, your volume gesture will fling the pointer across the screen. Section 9 fixes it by swallowing `mouseMoved` while an edge gesture is live.
- TCC permission is bound to your code signature.** If you sign ad-hoc (“Sign to Run Locally”), the signature changes on most builds and macOS will silently stop trusting the app — tap creation starts returning `nil` and you will think your code broke. Use a real Development certificate from day one. On macOS 27 this matters even more: Apple now lets users switch off Accessibility grants that were previously locked in by MDM, and the old PPC Accessibility key is deprecated.

Build order

We build in four milestones so that each one is independently testable:

- Milestone A** — a plain window that visualises your fingers on the trackpad. Proves the coordinate system and the zone maths. No permissions needed.
- Milestone B** — volume and brightness controllers, driven by test buttons.
- Milestone C** — the system-wide event tap; gestures work with any app frontmost.
- Milestone D** — menu-bar UI, per-edge configuration, persistence.

1. Set up the machine

1.1 Check your hardware

macOS 27 is **Apple silicon only** — every Intel Mac is off the list, and Xcode 27 will not install on one. Confirm with:

```
uname -m          # must print: arm64
sw_vers           # ProductVersion should be 27.x
```

1.2 Install or confirm macOS 27

macOS 27 Golden Gate reached general availability on 14 September 2026. If you are still on a developer beta build, switch to the release track:

1. **System Settings → General → Software Update**
2. Click the ⓘ next to *Beta Updates* and set it to **Off**
3. Install any pending update so you land on the shipping 27.0 build

Running the release build rather than a beta removes a whole class of “is this my bug or Apple’s?” questions, which matters a lot for event-tap and TCC work.

1.3 Install Xcode 27

Xcode 27 requires **macOS 26.4 or later** to run, and installs only on Apple silicon. Two routes:

Route A — Mac App Store (simplest). Search for Xcode, install. You get automatic updates.

Route B — developer.apple.com (pinned version). Sign in at developer.apple.com/download/applications/, grab the Xcode 27 .xip, double-click to expand (this takes 15–30 minutes), then drag `Xcode.app` into `/Applications`. Route B is better if you want to keep Xcode 26 side by side.

Then, in Terminal:

```
sudo xcode-select -s /Applications/Xcode.app/Contents/Developer
xcodebuild -version      # Xcode 27.x
swift --version          # Swift 6.4
```

Launch Xcode once and let it finish “Installing components”. Under **Xcode → Settings → Platforms**, confirm the **macOS 27** SDK is present.

1.4 Free Apple Developer account

You do **not** need the paid \$99 program to build and run EdgePad locally. A free Apple ID registered as a developer gives you a **Development** signing certificate, which is what keeps your code signature stable across builds — the thing that keeps Accessibility permission from resetting.

Xcode → Settings → Accounts → + → Apple ID, sign in, then select your team and click **Manage Certificates → + → Apple Development**.

You only need the paid program later, if you want to notarize EdgePad and hand it to other people.

2. Create the Xcode project

2.1 New project

1. **File → New → Project**
2. Platform **macOS**, template **App**, Next
3. Fill in:
 - Product Name: `EdgePad`
 - Team: your personal team
 - Organization Identifier: `com.yourname` → bundle ID becomes `com.yourname.EdgePad`
 - Interface: **SwiftUI**
 - Language: **Swift**
 - Testing System: None (add later if you like)
 - Storage: None
4. Save it somewhere outside iCloud Drive. iCloud sync plus DerivedData is a recipe for flaky builds.

2.2 Target settings

Select the project in the navigator, then the **EdgePad** target.

General tab - Minimum Deployments → macOS: set **26.0**. There is nothing in this project that needs a 27-only API, and a 26.0 floor keeps the app usable on Tahoe machines.

Signing & Capabilities tab - Signing Certificate: **Development** (not “Sign to Run Locally”). This is the step people skip; do not skip it. - Find the **App Sandbox** capability and **delete it** (hover, click the x). Event taps do not work inside the sandbox. - Leave **Hardened Runtime** enabled. It costs you nothing here and you will need it if you ever notarize.

Build Settings tab - Search for **Architectures**. Set **Architectures** to **arm64** only. macOS 27 is Apple silicon only, so a universal binary is dead weight. - Search for **Swift Language Version**. Swift 6 mode is the default in Xcode 27 and will flag concurrency issues in the event-tap code. Section 6 shows you how to satisfy it properly. If you would rather defer that fight, set **Swift Language Mode** to **5** for now and come back to it.

2.3 Info.plist keys

Still on the target, go to the **Info** tab and add:

Key	Type	Value
Application is agent (UIElement) (LSUIElement)	Boolean	YES

LSUIElement = YES removes the Dock icon and the menu bar, which is what you want for a background utility. Note that once you set it, **⌘R** will appear to launch nothing — the app is running, it just has no visible UI until Milestone D. Check Activity Monitor if you are unsure.

For Milestone A you may want to leave **LSUIElement** off so you get a normal window, and turn it on at Milestone D.

2.4 Project file layout

Create these groups and files now so the later sections have somewhere to land:

```
EdgePad/
├─ EdgePadApp.swift          (generated)
├─ AppDelegate.swift
├─ Core/
│  ├─ TrackpadEdge.swift
│  ├─ EdgeBinding.swift
│  └─ EdgeGestureRecognizer.swift
├─ Input/
│  ├─ TrackpadTouchView.swift
│  └─ GestureEventTap.swift
├─ Actions/
│  ├─ SystemVolume.swift
│  ├─ SystemBrightness.swift
│  └─ HIDKeyPoster.swift
├─ Permissions/
│  └─ PermissionChecker.swift
└─ UI/
   ├─ TouchDebugView.swift
   └─ PreferencesView.swift
```

3. Milestone A — see your fingers

Before any permissions or taps, prove that you can read touch coordinates at all.

3.1 The touch-capturing view

Input/TrackpadTouchView.swift :

```

import Cocoa

/// An NSView that receives indirect (trackpad) touches while the app is frontmost.
/// This is the "training wheels" input path – no permissions required.
final class TrackpadTouchView: NSView {

    /// Called on every touch phase change with the full current touch set.
    var onTouches: ((Set<NSTouch>) -> Void)?

    override init(frame frameRect: NSRect) {
        super.init(frame: frameRect)
        configure()
    }

    required init?(coder: NSCoder) {
        super.init(coder: coder)
        configure()
    }

    private func configure() {
        // .indirect == trackpad / Magic Mouse surface.
        // .direct would be a touchscreen, which Macs do not have.
        allowedTouchTypes = [.indirect]

        // Resting touches are fingers parked on the pad (e.g. a thumb near the
        // click zone). We do not want them counted as gestures.
        wantsRestingTouches = false
    }

    override var acceptsFirstResponder: Bool { true }

    override func touchesBegan(with event: NSEvent) { report(event) }
    override func touchesMoved(with event: NSEvent) { report(event) }
    override func touchesEnded(with event: NSEvent) { report(event) }
    override func touchesCancelled(with event: NSEvent) { report(event) }

    private func report(_ event: NSEvent) {
        onTouches?(event.touches(matching: .any, in: self))
    }
}

```

Key facts about `NSTouch` :

- `normalizedPosition` → `CGPoint` in `0...1`, **origin bottom-left**. `(1, 1)` is the top-right corner of the trackpad.
- `phase` → `.began`, `.moved`, `.stationary`, `.ended`, `.cancelled`.
- `identity` → an opaque object that is **the same instance** for one finger across its whole lifetime. This is how you follow a specific finger. Do not try to match fingers by position.
- `isResting` → true for a parked finger.
- `deviceSize` → physical size of the input surface in points. Useful to tell a Magic Mouse (small) from a trackpad (large) if you have both.

3.2 A visual debug view

`UI/TouchDebugView.swift` — a SwiftUI wrapper that draws your fingers as dots and shades the four edge bands, so you can literally see when a finger is “in the zone”:

```

import SwiftUI
import AppKit

struct TouchCapture: NSViewRepresentable {
    var onTouches: (Set<NSTouch>) -> Void
}

```

```

func makeNSView(context: Context) -> TrackpadTouchView {
    let view = TrackpadTouchView()
    view.onTouches = onTouches
    return view
}

func updateNSView(_ nsView: TrackpadTouchView, context: Context) {
    nsView.onTouches = onTouches
}

}

struct TouchDebugView: View {
    @State private var dots: [CGPoint] = []
    @State private var band: CGFloat = 0.15

    var body: some View {
        VStack(spacing: 12) {
            Text("Touch the trackpad. Dots follow your fingers.")
                .font(.callout)

            GeometryReader { geo in
                ZStack(alignment: .topLeading) {
                    Rectangle().fill(.quaternary)

                    // Edge bands
                    Rectangle().fill(.blue.opacity(0.18))
                        .frame(width: geo.size.width * band)
                        .frame(maxHeight: .infinity, alignment: .leading)
                    Rectangle().fill(.blue.opacity(0.18))
                        .frame(width: geo.size.width * band)
                        .frame(maxWidth: .infinity, maxHeight: .infinity,
                            alignment: .trailing)
                    Rectangle().fill(.green.opacity(0.18))
                        .frame(height: geo.size.height * band)
                        .frame(maxWidth: .infinity, alignment: .top)
                    Rectangle().fill(.green.opacity(0.18))
                        .frame(height: geo.size.height * band)
                        .frame(maxWidth: .infinity, maxHeight: .infinity,
                            alignment: .bottom)

                    ForEach(Array(dots.enumerated()), id: \.offset) { _, p in
                        Circle()
                            .fill(.orange)
                            .frame(width: 22, height: 22)
                            // NSTouch origin is bottom-left; SwiftUI is top-left,
                            // so flip y.
                            .position(x: p.x * geo.size.width,
                                y: (1 - p.y) * geo.size.height)
                    }
                }
            }
            .aspectRatio(1.6, contentMode: .fit)

            TouchCapture { touches in
                dots = touches
                    .filter { $0.phase != .ended && $0.phase != .cancelled }
                    .map(\.normalizedPosition)
            }
                .frame(height: 0) // invisible; it only needs to be in the responder chain
        }
        .padding()
        .frame(minWidth: 520, minHeight: 420)
    }
}

```

```
}  
}
```

Point `ContentView` at `TouchDebugView()`, run, and put two fingers on the trackpad. You should see two orange dots that track your fingers, and you should be able to see exactly when a dot enters the shaded band.

If nothing appears: the `TouchCapture` view has zero height, so it may never become first responder. Give it a real frame (say `.frame(height: 40)`) during debugging, or call `window?.makeFirstResponder(view)` from `viewDidMoveToWindow()`.

4. The zone model and the recognizer

Now that coordinates work, encode the logic. This code is shared by both the in-app path and the system-wide path, which is why it lives in `Core/`.

4.1 Edges

`Core/TrackpadEdge.swift`:

```
import Foundation

enum TrackpadEdge: String, CaseIterable, Codable, Identifiable {
    case left, right, top, bottom

    var id: String { rawValue }

    var displayName: String {
        switch self {
        case .left:    return "Left edge"
        case .right:   return "Right edge"
        case .top:     return "Top edge"
        case .bottom:  return "Bottom edge"
        }
    }

    /// Which coordinate changes as the finger slides *along* this edge.
    /// Vertical edges (left/right) are travelled by moving in y.
    /// Horizontal edges (top/bottom) are travelled by moving in x.
    var travelIsVertical: Bool {
        self == .left || self == .right
    }

    /// Position along the travel axis, 0...1.
    func travelValue(of point: CGPoint) -> CGFloat {
        travelIsVertical ? point.y : point.x
    }

    /// Distance from the edge itself, 0 at the edge, growing inward.
    func depth(of point: CGPoint) -> CGFloat {
        switch self {
        case .left:    return point.x
        case .right:   return 1 - point.x
        case .bottom:  return point.y
        case .top:     return 1 - point.y
        }
    }
}
```

4.2 Bindings

`Core/EdgeBinding.swift`:

```

import Foundation

enum EdgeAction: String, Codable, CaseIterable, Identifiable {
    case none
    case volume
    case brightness
    case keyboardBacklight
    case scroll
    case mediaScrub

    var id: String { rawValue }

    var displayName: String {
        switch self {
            case .none:           return "Nothing"
            case .volume:         return "Volume"
            case .brightness:     return "Display brightness"
            case .keyboardBacklight: return "Keyboard backlight"
            case .scroll:         return "Scroll"
            case .mediaScrub:     return "Track skip"
        }
    }
}

struct EdgeBinding: Codable, Equatable {
    var action: EdgeAction = .none

    /// How wide the active band is, as a fraction of the trackpad. 0.15 ≈ 15%.
    var bandThickness: CGFloat = 0.15

    /// How far the finger must travel (normalized) before one step fires.
    /// Smaller = more sensitive. 0.05 ≈ 20 steps across the whole pad.
    var stepDistance: CGFloat = 0.05

    /// Cancel the gesture if the finger wanders this far past the band.
    var escapeMargin: CGFloat = 0.08

    /// Flip the direction.
    var inverted: Bool = false

    var isEnabled: Bool { action != .none }
}

struct EdgeConfiguration: Codable, Equatable {
    var bindings: [TrackpadEdge: EdgeBinding]

    static let `default` = EdgeConfiguration(bindings: [
        .right: EdgeBinding(action: .volume),
        .top: EdgeBinding(action: .brightness),
        .left: EdgeBinding(action: .none),
        .bottom: EdgeBinding(action: .none),
    ])
}

```

4.3 The recognizer

This is the heart of the app. It is a small state machine, one entry per finger.

`Core/EdgeGestureRecognizer.swift`:

```

import AppKit

```

```

final class EdgeGestureRecognizer {

    var configuration: EdgeConfiguration = .default

    /// Fired when a step should happen. `direction` is +1 or -1.
    var onStep: ((TrackpadEdge, Int) -> Void)?

    /// True while at least one finger is mid-gesture. Used to suppress the cursor.
    private(set) var hasActiveGesture = false

    private struct Track {
        let edge: TrackpadEdge
        var lastTravel: CGFloat
        var accumulated: CGFloat = 0
    }

    private var tracks: [ObjectIdentifier: Track] = [:]

    // MARK: - Entry point

    func process(touches: Set<NSTouch>) {
        for touch in touches {
            guard touch.type == .indirect, !touch.isResting else { continue }
            handle(touch)
        }
        hasActiveGesture = !tracks.isEmpty
    }

    func reset() {
        tracks.removeAll()
        hasActiveGesture = false
    }

    // MARK: - Per-touch state machine

    private func handle(_ touch: NSTouch) {
        let key = ObjectIdentifier(touch.identity as AnyObject)
        let position = touch.normalizedPosition

        switch touch.phase {

        case .began:
            guard let edge = edge(containing: position),
                  let binding = configuration.bindings[edge],
                  binding.isEnabled
            else { return }
            tracks[key] = Track(edge: edge,
                               lastTravel: edge.travelValue(of: position))

        case .moved, .stationary:
            guard var track = tracks[key],
                  let binding = configuration.bindings[track.edge]
            else { return }

            // Bail out if the finger has drifted well inside the pad.
            if track.edge.depth(of: position)
                > binding.bandThickness + binding.escapeMargin {
                tracks[key] = nil
                return
            }

            let travel = track.edge.travelValue(of: position)

```

```

        track.accumulated += travel - track.lastTravel
        track.lastTravel = travel

        // Fire one step per stepDistance of travel. A while-loop rather than
        // an if, so a fast flick fires several steps instead of losing them.
        while abs(track.accumulated) >= binding.stepDistance {
            let raw = track.accumulated > 0 ? 1 : -1
            onStep?(track.edge, binding.inverted ? -raw : raw)
            track.accumulated -= CGFloat(raw) * binding.stepDistance
        }

        tracks[key] = track

        case .ended, .cancelled:
            tracks[key] = nil

        default:
            break
    }
}

// MARK: - Zone test

private func edge(containing p: CGPoint) -> TrackpadEdge? {
    // Corners are ambiguous. Resolve by picking whichever edge the finger is
    // closest to, among the edges it actually qualifies for.
    var best: (edge: TrackpadEdge, depth: CGFloat)?

    for edge in TrackpadEdge.allCases {
        guard let binding = configuration.bindings[edge],
              binding.isEnabled else { continue }
        let d = edge.depth(of: p)
        guard d <= binding.bandThickness else { continue }
        if best == nil || d < best!.depth {
            best = (edge, d)
        }
    }
    return best?.edge
}
}

```

Why `while` and not `if`: a quick flick can move the finger 0.3 normalized units between two event callbacks. With `if`, you would fire one step and throw away the rest, and fast gestures would feel dead. With `while`, a flick fires six steps.

Why `escapeMargin`: without it, a finger that starts at the right edge and sweeps diagonally into the middle keeps changing the volume. The margin gives a little hysteresis so the gesture does not die from tiny jitter, but does die when you clearly leave the zone.

5. Milestone B — the actions

5.1 Volume via Core Audio (precise)

Actions/SystemVolume.swift:

```

import Foundation
import CoreAudio
import AudioToolbox

enum SystemVolume {

    // MARK: - Device lookup

```

```

private static var defaultOutputDevice: AudioDeviceID? {
    var address = AudioObjectPropertyAddress(
        mSelector: kAudioHardwarePropertyDefaultOutputDevice,
        mScope:    kAudioObjectPropertyScopeGlobal,
        mElement:   kAudioObjectPropertyElementMain)

    var deviceID = AudioDeviceID(0)
    var size = UInt32(MemoryLayout<AudioDeviceID>.size)

    let status = AudioObjectGetPropertyData(
        AudioObjectID(kAudioObjectSystemObject),
        &address, 0, nil, &size, &deviceID)

    return status == noErr && deviceID != kAudioObjectUnknown ? deviceID : nil
}

private static var volumeAddress = AudioObjectPropertyAddress(
    mSelector: kAudioHardwareServiceDeviceProperty_VirtualMainVolume,
    mScope:    kAudioDevicePropertyScopeOutput,
    mElement:   kAudioObjectPropertyElementMain)

/// True if the current output device exposes a settable main volume.
/// Some HDMI and USB DACs do not.
static var isSupported: Bool {
    guard let device = defaultOutputDevice else { return false }
    var address = volumeAddress
    return AudioObjectHasProperty(device, &address)
}

/// MARK: - Read / write

static func current() -> Float? {
    guard let device = defaultOutputDevice else { return nil }
    var address = volumeAddress
    var value = Float32(0)
    var size = UInt32(MemoryLayout<Float32>.size)
    let status = AudioObjectGetPropertyData(
        device, &address, 0, nil, &size, &value)
    return status == noErr ? value : nil
}

@discardableResult
static func set(_ newValue: Float) -> Bool {
    guard let device = defaultOutputDevice else { return false }
    var address = volumeAddress
    var value = Float32(min(max(newValue, 0), 1))
    let status = AudioObjectSetPropertyData(
        device, &address, 0, nil,
        UInt32(MemoryLayout<Float32>.size), &value)
    if status == noErr && value > 0 { setMuted(false) }
    return status == noErr
}

static func setMuted(_ muted: Bool) {
    guard let device = defaultOutputDevice else { return }
    var address = AudioObjectPropertyAddress(
        mSelector: kAudioDevicePropertyMute,
        mScope:    kAudioDevicePropertyScopeOutput,
        mElement:   kAudioObjectPropertyElementMain)
    guard AudioObjectHasProperty(device, &address) else { return }
    var value: UInt32 = muted ? 1 : 0

```

```

        AudioObjectSetPropertyData(
            device, &address, 0, nil,
            UInt32(MemoryLayout<UInt32>.size), &value)
    }

    // MARK: - Stepping

    /// One sixteenth per step matches the granularity of the hardware volume keys.
    static func nudge(_ direction: Int, step: Float = 1.0 / 16.0) {
        if let value = current(), isSupported {
            set(value + Float(direction) * step)
        } else {
            // Fallback for devices with no virtual main volume.
            HIDKeyPoster.post(direction > 0 ? .soundUp : .soundDown)
        }
    }
}

```

Two notes. The constant is `kAudioHardwareServiceDeviceProperty_VirtualMainVolume` — Apple renamed the `Master` spellings to `Main` in macOS 12, and the old names are gone from the macOS 27 SDK. Same for `kAudioObjectPropertyElementMain`. Also, Core Audio changes the volume **silently**, with no on-screen HUD. If you want the HUD, use the media-key path below instead.

5.2 Media keys (gives you the system HUD)

`Actions/HIDKeyPoster.swift` :

```

import Cocoa

/// Synthesises the special keys on the top row of an Apple keyboard.
/// These are NX_KEYTYPE_* values from <IOKit/hidsystem/ev_keymap.h>.
enum MediaKey: Int32 {
    case soundUp          = 0
    case soundDown        = 1
    case brightnessUp     = 2
    case brightnessDown   = 3
    case mute              = 7
    case play              = 16
    case next              = 17
    case previous          = 18
    case illuminationUp    = 21
    case illuminationDown = 22
}

enum HIDKeyPoster {

    /// Post one full press (down then up).
    /// `fineGrained` adds Shift+Option, which most macOS HUDs interpret as
    /// quarter-steps instead of full steps.
    static func post(_ key: MediaKey, fineGrained: Bool = false) {
        send(key, isDown: true, fineGrained: fineGrained)
        send(key, isDown: false, fineGrained: fineGrained)
    }

    private static func send(_ key: MediaKey, isDown: Bool, fineGrained: Bool) {
        let state = isDown ? 0x0A : 0x0B
        var flagBits = UInt(state << 8)
        if fineGrained {
            flagBits |= NSEvent.ModifierFlags.shift.rawValue
            flagBits |= NSEvent.ModifierFlags.option.rawValue
        }

        let data1 = Int((key.rawValue << 16) | Int32(state << 8))

        guard let event = NSEvent.otherEvent(
            with: .systemDefined,
            location: .zero,
            modifierFlags: NSEvent.ModifierFlags(rawValue: flagBits),
            timestamp: ProcessInfo.processInfo.systemUptime,
            windowNumber: 0,
            context: nil,
            subtype: 8, // NX_SUBTYPE_AUX_CONTROL_BUTTONS
            data1: data1,
            data2: -1
        ) else { return }

        event.cgEvent?.post(tap: .cghidEventTap)
    }
}

```

Posting these events requires **Accessibility** permission. `subtype: 8` is the magic number that marks the event as an auxiliary-control (media key) event; it is the same value the real keyboard driver uses.

5.3 Brightness

There is no public, supported API to set display brightness on Apple silicon. The old IOKit route (`IODisplaySetFloatParameter`) does not work on internal Apple silicon displays, and `CoreDisplay_Display_SetUserBrightness` / `DisplayServicesSetBrightness` are private framework symbols that break between OS releases and block notarization for App Store distribution.

Use the media keys. They are synthetic input, not private API, they work on every display the system knows how to dim, and you get the brightness HUD for free.

Actions/SystemBrightness.swift:

```
import Foundation

enum SystemBrightness {
    /// `direction` is +1 or -1.
    static func nudge(_ direction: Int, fineGrained: Bool = false) {
        HIDKeyPoster.post(direction > 0 ? .brightnessUp : .brightnessDown,
                          fineGrained: fineGrained)
    }
}

enum KeyboardBacklight {
    static func nudge(_ direction: Int) {
        HIDKeyPoster.post(direction > 0 ? .illuminationUp : .illuminationDown)
    }
}
```

5.4 Test it before going further

Temporarily give `TouchDebugView` four buttons that call `SystemVolume.nudge(+1)`, `SystemVolume.nudge(-1)`, `SystemBrightness.nudge(+1)`, `SystemBrightness.nudge(-1)`. Run. Grant Accessibility when asked. If the buttons work, your actions layer is done and any later failure is in the input layer.

6. Milestone C — going system-wide with a CGEventTap

Milestone A only works when EdgePad is frontmost. To make gestures work anywhere, you install a tap into the session event stream.

6.1 Permissions

Permissions/PermissionChecker.swift:

```

import Cocoa
import ApplicationServices

enum PermissionChecker {

    /// Accessibility ("control your computer"). Required for event taps and for
    /// posting synthetic key events.
    static var hasAccessibility: Bool {
        AXIsProcessTrusted()
    }

    /// Shows the system prompt if not yet granted. Only prompts once per
    /// code-signing identity; after that the user must go to Settings manually.
    @discardableResult
    static func requestAccessibility() -> Bool {
        let key = kAXTrustedCheckOptionPrompt.takeUnretainedValue() as String
        return AXIsProcessTrustedWithOptions([key: true] as CFDictionary)
    }

    static func openAccessibilitySettings() {
        let url = URL(string:
            "x-apple.systempreferences:com.apple.preference.security?Privacy_Accessibility")!
        NSWorkspace.shared.open(url)
    }

    static func openInputMonitoringSettings() {
        let url = URL(string:
            "x-apple.systempreferences:com.apple.preference.security?Privacy_ListenEvent")!
        NSWorkspace.shared.open(url)
    }

    /// Poll until granted — there is no notification for TCC changes.
    static func waitForAccessibility(
        interval: TimeInterval = 1.0,
        onGranted: @escaping () -> Void
    ) {
        guard !hasAccessibility else { onGranted(); return }
        Timer.scheduledTimer(withTimeInterval: interval, repeats: true) { timer in
            if hasAccessibility {
                timer.invalidate()
                onGranted()
            }
        }
    }
}

```

On macOS 27 the user can revoke Accessibility at any time, including on MDM-managed machines where it used to be pinned. Never assume the grant persists — check `AXIsProcessTrusted()` on every wake and re-create the tap if it fails.

6.2 The tap

Input/GestureRecognizer.swift :

```

import Cocoa

/// Taps the session event stream for gesture (touch) events, plus mouseMoved so we
/// can suppress cursor drift during an edge gesture.
final class GestureEventTap: @unchecked Sendable {

    /// Return `true` from the handler to swallow the event.
    typealias Handler = (NSEvent, CGEventType) -> Bool
}

```

```

/// NSEvent.EventType.gesture has no CGEventType case. Its raw value is 29.
private static let gestureRawValue: UInt32 = 29

private var machPort: CFMachPort?
private var runLoopSource: CFRunLoopSource?
private let handler: Handler

init(handler: @escaping Handler) {
    self.handler = handler
}

// MARK: - Lifecycle

@discardableResult
func start() -> Bool {
    guard machPort == nil else { return true }

    let mask: CGEventMask =
        (1 << UInt64(Self.gestureRawValue))
        | (1 << UInt64(CGEventType.mouseMoved.rawValue))

    // Must be a bare C function – no captures.
    let callback: CGEventTapCallBack = { _, type, event, refcon in
        guard let refcon else { return Unmanaged.passUnretained(event) }
        let tap = Unmanaged<GestureEventTap>
            .fromOpaque(refcon)
            .takeUnretainedValue()
        return tap.process(type: type, event: event)
    }

    guard let port = CGEvent.tapCreate(
        tap: .cgSessionEventTap,
        place: .headInsertEventTap,
        options: .defaultTap,           // .listenOnly cannot swallow events
        eventsOfInterest: mask,
        callback: callback,
        userInfo: Unmanaged.passUnretained(self).toOpaque()
    ) else {
        return false                    // almost always a missing permission
    }

    machPort = port
    runLoopSource = CFMachPortCreateRunLoopSource(kCFAllocatorDefault, port, 0)
    CFRunLoopAddSource(CFRunLoopGetMain(), runLoopSource, .commonModes)
    CGEvent.tapEnable(tap: port, enable: true)
    return true
}

func stop() {
    if let source = runLoopSource {
        CFRunLoopRemoveSource(CFRunLoopGetMain(), source, .commonModes)
    }
    if let port = machPort {
        CGEvent.tapEnable(tap: port, enable: false)
        CFMachPortInvalidate(port)
    }
    runLoopSource = nil
    machPort = nil
}

// MARK: - Callback body

```

```

private func process(type: CGEventType,
                    event: CGEvent) -> Unmanaged<CGEvent>? {

    // The system disables slow taps. Re-arm and carry on.
    if type == .tapDisabledByTimeout || type == .tapDisabledByUserInput {
        if let port = machPort { CGEvent.tapEnable(tap: port, enable: true) }
        return Unmanaged.passUnretained(event)
    }

    guard let nsEvent = NSEvent(cgEvent: event) else {
        return Unmanaged.passUnretained(event)
    }

    return handler(nsEvent, type) ? nil : Unmanaged.passUnretained(event)
}
}

```

Things worth understanding here:

- `CGEvent.tapCreate` returns `nil` when you lack permission. It does not throw and it does not tell you why. Treat `nil` as “go check Accessibility, then Input Monitoring”.
- The callback runs on whatever run loop you attached the source to — here, the main run loop. That means your recognizer and action code are effectively on the main thread, so no locking needed.
- `.headInsertEventTap` puts you ahead of other observers, which you need if you intend to swallow events.
- The tap must return promptly. If your handler blocks, macOS disables the tap and sends `.tapDisabledByTimeout`, which is why the re-arm branch exists.
- `@unchecked Sendable` is the pragmatic answer to Swift 6 strict concurrency here. The class is only ever touched from the main run loop; the compiler cannot prove that through a C function pointer, so you assert it.

6.3 Reading touches out of a tapped event

This is the piece that makes the whole thing work:

```
let touches = nsEvent.allTouches()
```

`NSEvent.allTouches()` returns the full `Set<NSTouch>` and, importantly, works on an `NSEvent` reconstructed from a `CGEvent` — the touch payload survives the round trip. This is not spelled out in Apple’s documentation, so **verify it on the very first run**: log `touches.count` and confirm you get non-zero values.

If for some reason it returns an empty set on your machine, the fallback is:

```

let monitor = NSEvent.addGlobalMonitorForEvents(matching: [.gesture]) { event in
    let touches = event.allTouches()
    // ...
}

```

Global monitors are listen-only (you cannot swallow the cursor movement) but they are a supported API and need only Accessibility.

7. Wiring it together

`AppDelegate.swift`:

```

import Cocoa

final class AppDelegate: NSObject, NSApplicationDelegate {

    private let recognizer = EdgeGestureRecognizer()
    private var tap: GestureEventTap?

```

```

func applicationDidFinishLaunching(_ notification: Notification) {
    recognizer.configuration = ConfigurationStore.shared.load()
    recognizer.onStep = { [weak self] edge, direction in
        self?.perform(edge: edge, direction: direction)
    }

    if PermissionChecker.hasAccessibility {
        startTap()
    } else {
        PermissionChecker.requestAccessibility()
        PermissionChecker.waitForAccessibility { [weak self] in
            self?.startTap()
        }
    }

    // TCC can be revoked at any time; re-check after every wake.
    NSWorkspace.shared.notificationCenter.addObserver(
        forName: NSWorkspace.didWakeNotification,
        object: nil, queue: .main
    ) { [weak self] _ in
        self?.restartTapIfNeeded()
    }
}

func applicationWillTerminate(_ notification: Notification) {
    tap?.stop()
}

// MARK: - Tap

private func startTap() {
    let tap = GestureEventTap { [weak self] event, type in
        self?.handle(event: event, type: type) ?? false
    }
    if tap.start() {
        self.tap = tap
    } else {
        presentPermissionAlert()
    }
}

private func restartTapIfNeeded() {
    guard PermissionChecker.hasAccessibility else { return }
    tap?.stop()
    tap = nil
    recognizer.reset()
    startTap()
}

// MARK: - Event handling

private func handle(event: NSEvent, type: CGEventType) -> Bool {
    if type == .mouseMoved {
        // Swallow pointer movement while an edge gesture is in progress,
        // so adjusting the volume does not fling the cursor.
        return recognizer.hasActiveGesture
    }

    let touches = event.allTouches()
    guard !touches.isEmpty else { return false }
    recognizer.process(touches: touches)
    return false // never swallow the gesture event itself
}

```

```

    }

    // MARK: - Actions

    private func perform(edge: TrackpadEdge, direction: Int) {
        guard let binding = recognizer.configuration.bindings[edge] else { return }
        switch binding.action {
        case .none:
            break
        case .volume:
            SystemVolume.nudge(direction)
        case .brightness:
            SystemBrightness.nudge(direction)
        case .keyboardBacklight:
            KeyboardBacklight.nudge(direction)
        case .scroll:
            postScroll(direction)
        case .mediaScrub:
            HIDKeyPoster.post(direction > 0 ? .next : .previous)
        }
    }

    private func postScroll(_ direction: Int) {
        guard let event = CGEvent(scrollWheelEvent2Source: nil,
                                   units: .line,
                                   wheelCount: 1,
                                   wheel1: Int32(direction * 3),
                                   wheel2: 0, wheel3: 0) else { return }
        event.post(tap: .cghidEventTap)
    }

    private func presentPermissionAlert() {
        let alert = NSAlert()
        alert.messageText = "EdgePad needs permission"
        alert.informativeText = """
            EdgePad reads trackpad touches, which macOS treats as privileged input.

            Enable EdgePad under Privacy & Security → Accessibility. If gestures \
            still do not register, also add it under Input Monitoring.
            """
        alert.addButton(withTitle: "Open Accessibility")
        alert.addButton(withTitle: "Open Input Monitoring")
        alert.addButton(withTitle: "Later")

        switch alert.runModal() {
        case .alertFirstButtonReturn:
            PermissionChecker.openAccessibilitySettings()
        case .alertSecondButtonReturn:
            PermissionChecker.openInputMonitoringSettings()
        default:
            break
        }
    }
}

```

And the app entry point, `EdgePadApp.swift` :

```

import SwiftUI

@main
struct EdgePadApp: App {
    @NSApplicationDelegateAdaptor(AppDelegate.self) private var appDelegate

    var body: some Scene {
        MenuBarExtra("EdgePad", systemImage: "hand.point.up.left") {
            Button("Settings...") {
                NSApp.activate(ignoringOtherApps: true)
                NSApp.sendAction(Selector(("showSettingsWindow:")), to: nil, from: nil)
            }
            Divider()
            Button("Quit EdgePad") { NSApplication.shared.terminate(nil) }
                .keyboardShortcut("q")
        }

        Settings {
            PreferencesView()
        }
    }
}

```

At this point, build, run, grant Accessibility, quit, relaunch. Switch to Safari and slide a finger up the right edge of the trackpad. The volume should move.

8. First run: the permission dance

This trips everyone up, so follow it exactly.

1. Build and run from Xcode once. The app appears in the menu bar.
2. The permission alert appears. Click **Open Accessibility**.
3. In **System Settings → Privacy & Security → Accessibility**, find EdgePad and switch it on. If it is not listed, drag the built app in from `~/Library/Developer/Xcode/DerivedData/EdgePad-*/Build/Products/Debug/EdgePad.app` using the **+** button.
4. **Quit EdgePad completely and relaunch.** Accessibility grants are read at process start. An already-running process does not pick up a fresh grant.
5. If gestures still do not fire, open **Privacy & Security → Input Monitoring** and add EdgePad there too.

Strongly recommended: copy the built `.app` into `/Applications` and test from there rather than from DerivedData. DerivedData paths change, and TCC keys entries partly by path. Testing from `/Applications` eliminates a whole category of “it worked yesterday” bugs.

If macOS keeps asking for permission on every build: your signing identity is unstable. Go back to Signing & Capabilities and make sure it says *Development* with your team, not *Sign to Run Locally*. Then clear the stale entries:

```

tccutil reset Accessibility com.yourname.EdgePad
tccutil reset ListenEvent com.yourname.EdgePad

```

9. Living with the cursor

Even with `mouseMoved` suppression, a single finger on the trackpad is doing two jobs at once. Three strategies, in increasing order of polish:

Strategy 1 — suppress `mouseMoved` (implemented above). Cheap and mostly works. The cursor freezes for the duration of the gesture and resumes where it was. Downside: there is a one-event lag between the touch starting and the recognizer knowing about it, so you may see a few pixels of drift at the start.

Strategy 2 — require the gesture to begin at rest. Track how long the finger has been in the band before you start accumulating. If the finger arrived from the middle of the pad at speed, it is probably a normal pointer movement, not an edge gesture. Add a `minimumDwell` of about 80 ms to `EdgeBinding`.

Strategy 3 — require a specific finger count. Two fingers in the right-edge band is unambiguous — it cannot be confused with pointer movement. The cost is that it collides with two-finger scrolling, so you would also need to swallow `.scrollWheel` events while the gesture is live. Add a `requiredTouchCount` to `EdgeBinding` and count how many active tracks share the same edge.

A reasonable default is Strategy 1 plus Strategy 2.

10. Milestone D — configuring all four edges

10.1 Persistence

Core/ConfigurationStore.swift :

```
import Foundation

final class ConfigurationStore {
    static let shared = ConfigurationStore()
    private let key = "edgeConfiguration"

    func load() -> EdgeConfiguration {
        guard let data = UserDefaults.standard.data(forKey: key),
              let decoded = try? JSONDecoder()
                .decode(EdgeConfiguration.self, from: data)
        else { return .default }
        return decoded
    }

    func save(_ configuration: EdgeConfiguration) {
        guard let data = try? JSONEncoder().encode(configuration) else { return }
        UserDefaults.standard.set(data, forKey: key)
    }
}
```

`EdgeConfiguration` uses `[TrackpadEdge: EdgeBinding]`. Swift encodes dictionaries with non-`String` / `Int` keys as a flat array, which is ugly but functional. Since `TrackpadEdge` is `String`-backed, add `Codable` conformance via `RawRepresentable` to get clean JSON, or simply store four named properties instead of a dictionary if you would rather keep it boring.

10.2 Preferences UI

UI/PreferencesView.swift :

```
import SwiftUI

struct PreferencesView: View {
    @State private var configuration = ConfigurationStore.shared.load()
    @State private var selection: TrackpadEdge = .right

    var body: some View {
        HSplitView {
            List(TrackpadEdge.allCases, selection: $selection) { edge in
                Label(edge.displayName, systemImage: icon(for: edge))
                    .tag(edge)
            }
            .frame(minWidth: 180)

            if let binding = configuration.bindings[selection] {
                form(for: selection, binding: binding)
                    .frame(minWidth: 340)
            }
        }
        .frame(minWidth: 560, minHeight: 360)
    }
}
```

```

        .onChange(of: configuration) { _, new in
            ConfigurationStore.shared.save(new)
            NotificationCenter.default.post(
                name: .edgeConfigurationChanged, object: new)
        }
    }

    private func form(for edge: TrackpadEdge, binding: EdgeBinding) -> some View {
        Form {
            Picker("Action", selection: bind(edge, \.action)) {
                ForEach(EdgeAction.allCases) { action in
                    Text(action.displayName).tag(action)
                }
            }

            Toggle("Invert direction", isOn: bind(edge, \.inverted))

            VStack(alignment: .leading) {
                Text("Band width: \(Int(binding.bandThickness * 100))%")
                Slider(value: bind(edge, \.bandThickness), in: 0.05...0.35)
            }

            VStack(alignment: .leading) {
                Text("Sensitivity")
                Slider(value: bind(edge, \.stepDistance), in: 0.01...0.20)
                Text("Smaller steps = more sensitive")
                    .font(.caption).foregroundColor(.secondary)
            }
        }
        .formStyle(.grouped)
        .disabled(binding.action == .none && false)
    }

    private func bind<V>(_ edge: TrackpadEdge,
                        _ path: WritableKeyPath<EdgeBinding, V>) -> Binding<V> {
        Binding(
            get: { configuration.bindings[edge]![keyPath: path] },
            set: { configuration.bindings[edge]![keyPath: path] = $0 }
        )
    }

    private func icon(for edge: TrackpadEdge) -> String {
        switch edge {
        case .left:    return "arrow.left.to.line"
        case .right:   return "arrow.right.to.line"
        case .top:     return "arrow.up.to.line"
        case .bottom:  return "arrow.down.to.line"
        }
    }
}

extension Notification.Name {
    static let edgeConfigurationChanged =
        Notification.Name("edgeConfigurationChanged")
}

```

Have `AppDelegate` observe `.edgeConfigurationChanged` and assign the new value to `recognizer.configuration`.

10.3 Launch at login

```
import ServiceManagement

enum LaunchAtLogin {
    static var isEnabled: Bool {
        SMAAppService.mainApp.status == .enabled
    }

    static func set(_ enabled: Bool) throws {
        if enabled {
            try SMAAppService.mainApp.register()
        } else {
            try SMAAppService.mainApp.unregister()
        }
    }
}
```

`SMAAppService` requires the app to be in `/Applications` and properly signed. It will throw otherwise.

11. Troubleshooting

Symptom	Likely cause	Fix
<code>tapCreate</code> returns <code>nil</code>	No Accessibility grant, or the process was launched before the grant	Grant, then fully quit and relaunch
Permission resets on every build	Ad-hoc signature	Switch to a Development certificate; <code>tccutil reset</code> the old entries
App is listed and enabled but tap still fails	Stale TCC row pointing at an old binary path	Remove the entry with <code>—</code> , re-add the app from <code>/Applications</code>
<code>allTouches()</code> returns empty	Event is not a gesture event, or the Magic Mouse/trackpad is disconnected	Log <code>nsEvent.type.rawValue</code> ; you want 29
Gestures fire, then stop after a few seconds	Tap disabled by timeout because the handler was slow	Keep the handler trivial; never do I/O or <code>print</code> inside it
Volume does nothing, brightness works	Output device has no virtual main volume (common on HDMI/USB DACs)	The <code>isSupported</code> fallback in <code>SystemVolume</code> covers this
Brightness does nothing on an external display	The display does not report brightness control to macOS	External DDC monitors need a separate DDC/CI implementation; out of scope
Steps feel jumpy or too fast	<code>stepDistance</code> too small	Raise it to 0.08–0.12 and retest
Cursor still drifts during a gesture	<code>options:</code> is <code>.listenOnly</code>	Must be <code>.defaultTap</code> to swallow events
Compiler complains about <code>Sendable</code> in the tap	Swift 6 strict concurrency	Keep <code>@unchecked Sendable</code> on <code>GestureEventTap</code> , or set Swift Language Mode to 5
Nothing happens when you press <code>⌘R</code>	<code>LSUIElement</code> is on and there is no menu bar item yet	Expected until Milestone D; check Activity Monitor

A useful debugging habit

Add a temporary HUD that prints the live touch state to a floating window rather than to the console. `print` inside an event-tap callback is slow enough to trigger the timeout, which makes the bug you are hunting disappear. Write to a `@Published` property and render it instead.

12. Where to take it next

- **Per-app profiles.** Watch `NSWorkspace.didActivateApplicationNotification` and swap `configuration` based on the frontmost bundle identifier. Volume on the right edge in Music, timeline scrubbing in Final Cut.
- **Corner zones.** Treat the four corners as a fifth through eighth zone with tap actions rather than slides.
- **Acceleration.** Scale `stepDistance` by finger velocity so a slow drag is precise and a fast flick covers the whole range.

- **Haptic feedback.** `NSHapticFeedbackManager.defaultPerformer.perform(.levelChange, performanceTime: .now)` on each step makes the gesture feel like a real control. This is the single highest-impact polish item.
- **Custom HUD.** A borderless `NSPanel` with `.statusBar` window level showing the current value, so Core Audio volume changes get visual feedback too.
- **Distribution.** Developer ID signing + `notarytool` submission, wrapped in a DMG. Remember: the App Store is not an option while the app uses an event tap.

Appendix A — API reference card

What	API	Framework	Notes
Touch position	<code>NSTouch.normalizedPosition</code>	AppKit	<code>0...1</code> , origin bottom-left
Finger identity	<code>NSTouch.identity</code>	AppKit	Same object for a finger's lifetime
In-app touches	<code>NSView.touchesBegan(with:)</code>	AppKit	Needs <code>allowedTouchTypes = [.indirect]</code>
System-wide touches	<code>CGEvent.tapCreate</code> + <code>NSEvent.allTouches()</code>	CoreGraphics + AppKit	Gesture event raw type is 29
Permission check	<code>AXIsProcessTrusted()</code>	ApplicationServices	Read at process start
Volume read/write	<code>kAudioHardwareServiceDeviceProperty_VirtualMainVolume</code>	CoreAudio	No HUD
Mute	<code>kAudioDevicePropertyMute</code>	CoreAudio	
Media keys	<code>NSEvent.otherEvent(with: .systemDefined ...)</code> , subtype 8	AppKit	Needs Accessibility
Brightness	<code>NX_KEYTYPE_BRIGHTNESS_UP/DOWN</code> (2 / 3)	via media keys	Only supported route on Apple silicon
Scroll injection	<code>CGEvent(scrollWheelEvent2Source:...)</code>	CoreGraphics	
Haptics	<code>NSHapticFeedbackManager</code>	AppKit	Trackpad only
Launch at login	<code>SMAppService.mainApp</code>	ServiceManagement	App must live in <code>/Applications</code>

Appendix B — environment checklist

[] Apple silicon Mac	<code>uname -m → arm64</code>
[] macOS 27.x release build	<code>sw_vers</code>
[] Xcode 27 installed	<code>xcodebuild -version</code>
[] Swift 6.4 toolchain	<code>swift --version</code>
[] Free developer account added	Xcode → Settings → Accounts
[] Apple Development certificate	Manage Certificates
[] App Sandbox REMOVED	Signing & Capabilities
[] Hardened Runtime enabled	Signing & Capabilities
[] Architectures = arm64	Build Settings
[] Deployment target = macOS 26.0 General	
[] LSUIElement = YES	Info
[] Built app copied to <code>/Applications</code> for permission testing	